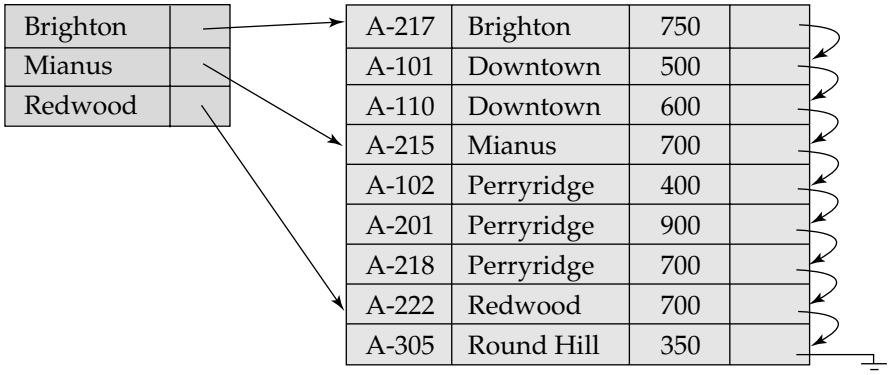


Figure 12.3 Sparse index.

cessing a database request is the time that it takes to bring a block from disk into main memory. Once we have brought in the block, the time to scan the entire block is negligible. Using this sparse index, we locate the block containing the record that we are seeking. Thus, unless the record is on an overflow block (see Section 11.7.1), we minimize block accesses while keeping the size of the index (and thus, our space overhead) as small as possible.

For the preceding technique to be fully general, we must consider the case where records for one search-key value occupy several blocks. It is easy to modify our scheme to handle this situation.

12.2.1.2 Multilevel Indices

Even if we use a sparse index, the index itself may become too large for efficient processing. It is not unreasonable, in practice, to have a file with 100,000 records, with 10 records stored in each block. If we have one index record per block, the index has 10,000 records. Index records are smaller than data records, so let us assume that 100 index records fit on a block. Thus, our index occupies 100 blocks. Such large indices are stored as sequential files on disk.

If an index is sufficiently small to be kept in main memory, the search time to find an entry is low. However, if the index is so large that it must be kept on disk, a search for an entry requires several disk block reads. Binary search can be used on the index file to locate an entry, but the search still has a large cost. If the index occupies b blocks, binary search requires as many as $\lceil \log_2(b) \rceil$ blocks to be read. ($\lceil x \rceil$ denotes the least integer that is greater than or equal to x; that is, we round upward.) For our 100-block index, binary search requires seven block reads. On a disk system where a block read takes 30 milliseconds, the search will take 210 milliseconds, which is long. Note that, if overflow blocks have been used, binary search will not be possible. In that case, a sequential search is typically used, and that requires b block reads, which will take even longer. Thus, the process of searching a large index may be costly.

To deal with this problem, we treat the index just as we would treat any other sequential file, and construct a sparse index on the primary index, as in Figure 12.4.

To locate a record, we first use binary search on the outer index to find the record for the largest search-key value less than or equal to the one that we desire. The pointer points to a block of the inner index. We scan this block until we find the record that has the largest search-key value less than or equal to the one that we desire. The pointer in this record points to the block of the file that contains the record for which we are looking.

Using the two levels of indexing, we have read only one index block, rather than the seven we read with binary search, if we assume that the outer index is already in main memory. If our file is extremely large, even the outer index may grow too large to fit in main memory. In such a case, we can create yet another level of index. Indeed, we can repeat this process as many times as necessary. Indices with two or more levels are called **multilevel** indices. Searching for records with a multilevel index requires significantly fewer I/O operations than does searching for records by binary search. Each level of index could correspond to a unit of physical storage. Thus, we may have indices at the track, cylinder, and disk levels.

A typical dictionary is an example of a multilevel index in the nondatabase world. The header of each page lists the first word alphabetically on that page. Such a book

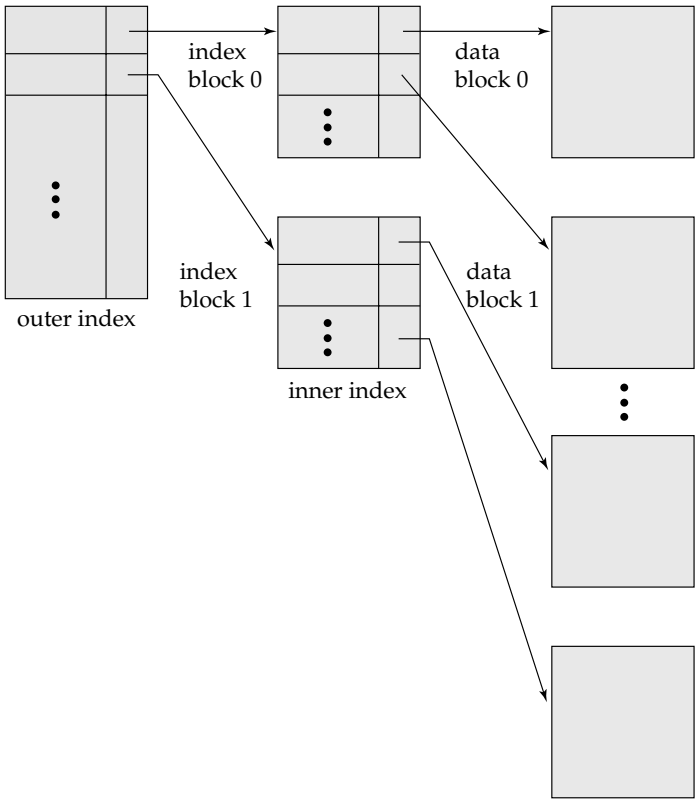


Figure 12.4 Two-level sparse index.

index is a multilevel index: The words at the top of each page of the book index form a sparse index on the contents of the dictionary pages.

Multilevel indices are closely related to tree structures, such as the binary trees used for in-memory indexing. We shall examine the relationship later, in Section 12.3.

12.2.1.3 Index Update

Regardless of what form of index is used, every index must be updated whenever a record is either inserted into or deleted from the file. We first describe algorithms for updating single-level indices.

- **Insertion**. First, the system performs a lookup using the search-key value that appears in the record to be inserted. Again, the actions the system takes next depend on whether the index is dense or sparse:
  - Dense indices:
    1. If the search-key value does not appear in the index, the system inserts an index record with the search-key value in the index at the appropriate position.
    2. Otherwise the following actions are taken:
       a. If the index record stores pointers to all records with the same search-key value, the system adds a pointer to the new record to the index record.
       b. Otherwise, the index record stores a pointer to only the first record with the search-key value. The system then places the record being inserted after the other records with the same search-key values.
  - Sparse indices: We assume that the index stores an entry for each block. If the system creates a new block, it inserts the first search-key value (in search-key order) appearing in the new block into the index. On the other hand, if the new record has the least search-key value in its block, the system updates the index entry pointing to the block; if not, the system makes no change to the index.
- **Deletion**. To delete a record, the system first looks up the record to be deleted. The actions the system takes next depend on whether the index is dense or sparse:
  - Dense indices:
    1. If the deleted record was the only record with its particular search-key value, then the system deletes the corresponding index record from the index.
    2. Otherwise the following actions are taken:
       a. If the index record stores pointers to all records with the same search-key value, the system deletes the pointer to the deleted record from the index record.

b. Otherwise, the index record stores a pointer to only the first record with the search-key value. In this case, if the deleted record was the first record with the search-key value, the system updates the index record to point to the next record.

- Sparse indices:
  1. If the index does not contain an index record with the search-key value of the deleted record, nothing needs to be done to the index.
  2. Otherwise the system takes the following actions:
     a. If the deleted record was the only record with its search key, the system replaces the corresponding index record with an index record for the next search-key value (in search-key order). If the next search-key value already has an index entry, the entry is deleted instead of being replaced.
     b. Otherwise, if the index record for the search-key value points to the record being deleted, the system updates the index record to point to the next record with the same search-key value.

Insertion and deletion algorithms for multilevel indices are a simple extension of the scheme just described. On deletion or insertion, the system updates the lowest-level index as described. As far as the second level is concerned, the lowest-level index is merely a file containing records—thus, if there is any change in the lowest-level index, the system updates the second-level index as described. The same technique applies to further levels of the index, if there are any.

12.2.2 Secondary Indices

Secondary indices must be dense, with an index entry for every search-key value, and a pointer to every record in the file. A primary index may be sparse, storing only some of the search-key values, since it is always possible to find records with intermediate search-key values by a sequential access to a part of the file, as described earlier. If a secondary index stores only some of the search-key values, records with intermediate search-key values may be anywhere in the file and, in general, we cannot find them without searching the entire file.

A secondary index on a candidate key looks just like a dense primary index, except that the records pointed to by successive values in the index are not stored sequentially. In general, however, secondary indices may have a different structure from primary indices. If the search key of a primary index is not a candidate key, it suffices if the index points to the first record with a particular value for the search key, since the other records can be fetched by a sequential scan of the file.

In contrast, if the search key of a secondary index is not a candidate key, it is not enough to point to just the first record with each search-key value. The remaining records with the same search-key value could be anywhere in the file, since the records are ordered by the search key of the primary index, rather than by the search key of the secondary index. Therefore, a secondary index must contain pointers to all the records.

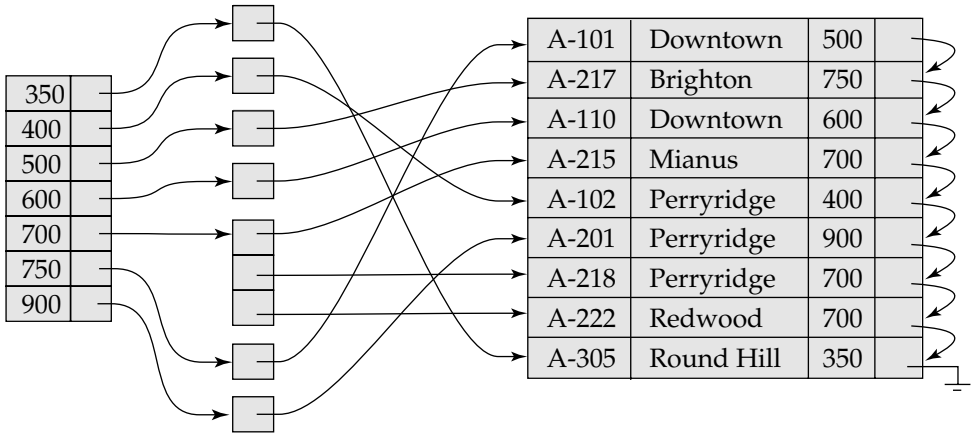


Figure 12.5 Secondary index on *account* file, on noncandidate key *balance*.

We can use an extra level of indirection to implement secondary indices on search keys that are not candidate keys. The pointers in such a secondary index do not point directly to the file. Instead, each points to a bucket that contains pointers to the file. Figure 12.5 shows the structure of a secondary index that uses an extra level of indirection on the *account* file, on the search key *balance*.

A sequential scan in primary index order is efficient because records in the file are stored physically in the same order as the index order. However, we cannot (except in rare special cases) store a file physically ordered both by the search key of the primary index, and the search key of a secondary index. Because secondary-key order and physical-key order differ, if we attempt to scan the file sequentially in secondary-key order, the reading of each record is likely to require the reading of a new block from disk, which is very slow.

The procedure described earlier for deletion and insertion can also be applied to secondary indices; the actions taken are those described for dense indices storing a pointer to every record in the file. If a file has multiple indices, whenever the file is modified, *every* index must be updated.

Secondary indices improve the performance of queries that use keys other than the search key of the primary index. However, they impose a significant overhead on modification of the database. The designer of a database decides which secondary indices are desirable on the basis of an estimate of the relative frequency of queries and modifications.

12.3 B+-Tree Index Files

The main disadvantage of the index-sequential file organization is that performance degrades as the file grows, both for index lookups and for sequential scans through the data. Although this degradation can be remedied by reorganization of the file, frequent reorganizations are undesirable.

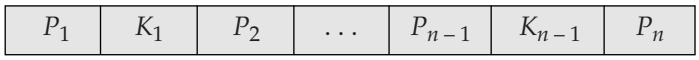


Figure 12.6 Typical node of a B$^+$-tree.

The **B$^+$-tree** index structure is the most widely used of several index structures that maintain their efficiency despite insertion and deletion of data. A B$^+$-tree index takes the form of a **balanced tree** in which every path from the root of the tree to a leaf of the tree is of the same length. Each nonleaf node in the tree has between $\lceil n/2 \rceil$ and n children, where n is fixed for a particular tree.

We shall see that the B$^+$-tree structure imposes performance overhead on insertion and deletion, and adds space overhead. The overhead is acceptable even for frequently modified files, since the cost of file reorganization is avoided. Furthermore, since nodes may be as much as half empty (if they have the minimum number of children), there is some wasted space. This space overhead, too, is acceptable given the performance benefits of the B$^+$-tree structure.

12.3.1 Structure of a B$^+$-Tree

A B$^+$-tree index is a multilevel index, but it has a structure that differs from that of the multilevel index-sequential file. Figure 12.6 shows a typical node of a B$^+$-tree. It contains up to $n - 1$ search-key values $K_1, K_2, \ldots, K_{n-1}$, and n pointers $P_1, P_2, \ldots, P_n$. The search-key values within a node are kept in sorted order; thus, if $i < j$, then $K_i < K_j$.

We consider first the structure of the leaf nodes. For $i = 1, 2, \ldots, n - 1$, pointer P_i points to either a file record with search-key value K_i or to a bucket of pointers, each of which points to a file record with search-key value K_i. The bucket structure is used only if the search key does not form a primary key, and if the file is not sorted in the search-key value order. Pointer P_n has a special purpose that we shall discuss shortly.

Figure 12.7 shows one leaf node of a B$^+$-tree for the *account* file, in which we have chosen n to be 3, and the search key is *branch-name*. Note that, since the account file is ordered by *branch-name*, the pointers in the leaf node point directly to the file.

Now that we have seen the structure of a leaf node, let us consider how search-key values are assigned to particular nodes. Each leaf can hold up to $n - 1$ values. We allow leaf nodes to contain as few as $\lceil (n-1)/2 \rceil$ values. The ranges of values in each leaf do not overlap. Thus, if L_i and L_j are leaf nodes and $i < j$, then every search-key value in L_i is less than every search-key value in L_j. If the B$^+$-tree index is to be a dense index, every search-key value must appear in some leaf node.

Now we can explain the use of the pointer P_n. Since there is a linear order on the leaves based on the search-key values that they contain, we use P_n to chain together the leaf nodes in search-key order. This ordering allows for efficient sequential processing of the file.

The nonleaf nodes of the B$^+$-tree form a multilevel (sparse) index on the leaf nodes. The structure of nonleaf nodes is the same as that for leaf nodes, except that all pointers are pointers to tree nodes. A nonleaf node may hold up to n pointers, and *must*

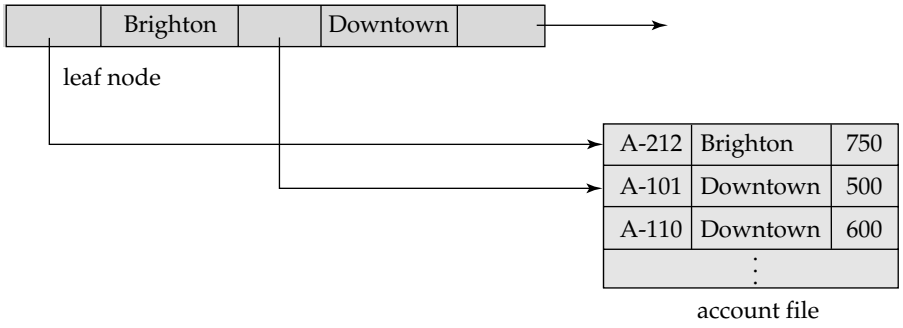


Figure 12.7 A leaf node for *account* B+-tree index ($n = 3$).

hold at least $\lceil n/2 \rceil$ pointers. The number of pointers in a node is called the *fanout* of the node.

Let us consider a node containing m pointers. For $i = 2, 3, \ldots, m - 1$, pointer P_i points to the subtree that contains search-key values less than K_i and greater than or equal to K_{i-1}. Pointer P_m points to the part of the subtree that contains those key values greater than or equal to K_{m-1}, and pointer P_1 points to the part of the subtree that contains those search-key values less than K_1.

Unlike other nonleaf nodes, the root node can hold fewer than $\lceil n/2 \rceil$ pointers; however, it must hold at least two pointers, unless the tree consists of only one node. It is always possible to construct a B+-tree, for any n, that satisfies the preceding requirements. Figure 12.8 shows a complete B+-tree for the *account* file ($n = 3$). For simplicity, we have omitted both the pointers to the file itself and the null pointers. As an example of a B+-tree for which the root must have less than $\lceil n/2 \rceil$ values, Figure 12.9 shows a B+-tree for the *account* file with $n = 5$.

These examples of B+-trees are all balanced. That is, the length of every path from the root to a leaf node is the same. This property is a requirement for a B+-tree. Indeed, the "B" in B+-tree stands for "balanced." It is the balance property of B+-trees that ensures good performance for lookup, insertion, and deletion.

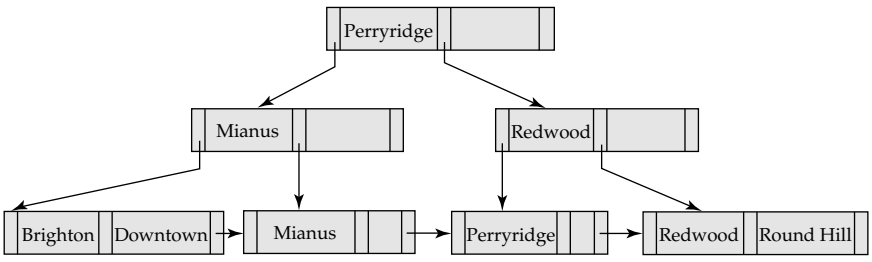


Figure 12.8 B+-tree for *account* file ($n = 3$).

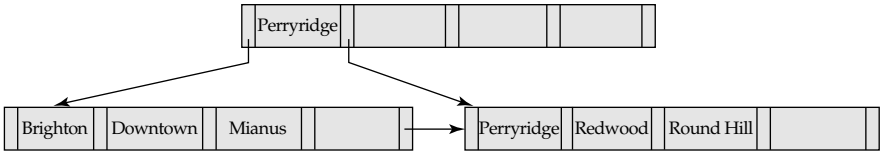


Figure 12.9 B$^+$-tree for *account* file with $n = 5$.

12.3.2 Queries on B$^+$-Trees

Let us consider how we process queries on a B$^+$-tree. Suppose that we wish to find all records with a search-key value of V. Figure 12.10 presents pseudocode for doing so. Intuitively, the procedure works as follows. First, we examine the root node, looking for the smallest search-key value greater than V. Suppose that we find that this search-key value is K_i. We then follow pointer P_i to another node. If we find no such value, then $k \geq K_{m-1}$, where m is the number of pointers in the node. In this case we follow P_m to another node. In the node we reached above, again we look for the smallest search-key value greater than V, and once again follow the corresponding pointer as above. Eventually, we reach a leaf node. At the leaf node, if we find search-key value K_i equals V, then pointer P_i directs us to the desired record or bucket. If the value V is not found in the leaf node, no record with key value V exists.

Thus, in processing a query, we traverse a path in the tree from the root to some leaf node. If there are K search-key values in the file, the path is no longer than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$.

In practice, only a few nodes need to be accessed, Typically, a node is made to be the same size as a disk block, which is typically 4 kilobytes. With a search-key size of 12 bytes, and a disk-pointer size of 8 bytes, n is around 200. Even with a more conservative estimate of 32 bytes for the search-key size, n is around 100. With $n = 100$, if we have 1 million search-key values in the file, a lookup requires only

```
procedure find(value V)
    set C = root node
    while C is not a leaf node begin
        Let K_i = smallest search-key value, if any, greater than V
        if there is no such value then begin
            Let m = the number of pointers in the node
            set C = node pointed to by P_m
        end
            else set C = the node pointed to by P_i
    end
    if there is a key value K_i in C such that K_i = V
        then pointer P_i directs us to the desired record or bucket
        else no record with key value k exists
end procedure
```

Figure 12.10 Querying a B$^+$-tree.

$\lceil \log_{50}(1{,}000{,}000) \rceil = 4$ nodes to be accessed. Thus, at most four blocks need to be read from disk for the lookup. The root node of the tree is usually heavily accessed and is likely to be in the buffer, so typically only three or fewer blocks need to be read from disk.

An important difference between B$^+$-tree structures and in-memory tree structures, such as binary trees, is the size of a node, and as a result, the height of the tree. In a binary tree, each node is small, and has at most two pointers. In a B$^+$-tree, each node is large—typically a disk block—and a node can have a large number of pointers. Thus, B$^+$-trees tend to be fat and short, unlike thin and tall binary trees. In a balanced binary tree, the path for a lookup can be of length $\lceil \log_2(K) \rceil$, where K is the number of search-key values. With $K = 1{,}000{,}000$ as in the previous example, a balanced binary tree requires around 20 node accesses. If each node were on a different disk block, 20 block reads would be required to process a lookup, in contrast to the four block reads for the B$^+$-tree.

12.3.3 Updates on B$^+$-Trees

Insertion and deletion are more complicated than lookup, since it may be necessary to **split** a node that becomes too large as the result of an insertion, or to **coalesce** nodes (that is, combine nodes) if a node becomes too small (fewer than $\lceil n/2 \rceil$ pointers). Furthermore, when a node is split or a pair of nodes is combined, we must ensure that balance is preserved. To introduce the idea behind insertion and deletion in a B$^+$-tree, we shall assume temporarily that nodes never become too large or too small. Under this assumption, insertion and deletion are performed as defined next.

- **Insertion**. Using the same technique as for lookup, we find the leaf node in which the search-key value would appear. If the search-key value already appears in the leaf node, we add the new record to the file and, if necessary, add to the bucket a pointer to the record. If the search-key value does not appear, we insert the value in the leaf node, and position it such that the search keys are still in order. We then insert the new record in the file and, if necessary, create a new bucket with the appropriate pointer.
- **Deletion**. Using the same technique as for lookup, we find the record to be deleted, and remove it from the file. We remove the search-key value from the leaf node if there is no bucket associated with that search-key value or if the bucket becomes empty as a result of the deletion.

We now consider an example in which a node must be split. Assume that we wish to insert a record with a *branch-name* value of "Clearview" into the B$^+$-tree of Figure 12.8. Using the algorithm for lookup, we find that "Clearview" should appear in the node containing "Brighton" and "Downtown." There is no room to insert the search-key value "Clearview." Therefore, the node is *split* into two nodes. Figure 12.11 shows the two leaf nodes that result from inserting "Clearview" and splitting the node containing "Brighton" and "Downtown." In general, we take the n search-key

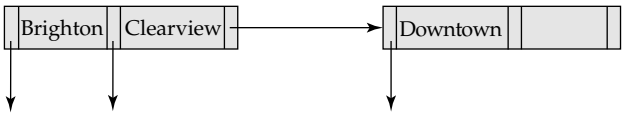


Figure 12.11 Split of leaf node on insertion of "Clearview."

values (the $n - 1$ values in the leaf node plus the value being inserted), and put the first $\lceil n/2 \rceil$ in the existing node and the remaining values in a new node.

Having split a leaf node, we must insert the new leaf node into the B$^+$-tree structure. In our example, the new node has "Downtown" as its smallest search-key value. We need to insert this search-key value into the parent of the leaf node that was split. The B$^+$-tree of Figure 12.12 shows the result of the insertion. The search-key value "Downtown" was inserted into the parent. It was possible to perform this insertion because there was room for an added search-key value. If there were no room, the parent would have had to be split. In the worst case, all nodes along the path to the root must be split. If the root itself is split, the entire tree becomes deeper.

The general technique for insertion into a B$^+$-tree is to determine the leaf node l into which insertion must occur. If a split results, insert the new node into the parent of node l. If this insertion causes a split, proceed recursively up the tree until either an insertion does not cause a split or a new root is created.

Figure 12.13 outlines the insertion algorithm in pseudocode. In the pseudocode, $L.K_i$ and $L.P_i$ denote the ith value and the ith pointer in node L, respectively. The pseudocode also makes use of the function $parent(L)$ to find the parent of a node L. We can compute a list of nodes in the path from the root to the leaf while initially finding the leaf node, and can use it later to find the parent of any node in the path efficiently. The pseudocode refers to inserting an entry (V, P) into a node. In the case of leaf nodes, the pointer to an entry actually precedes the key value, so the leaf node actually stores P before V. For internal nodes, P is stored just after V.

We now consider deletions that cause tree nodes to contain too few pointers. First, let us delete "Downtown" from the B$^+$-tree of Figure 12.12. We locate the entry for "Downtown" by using our lookup algorithm. When we delete the entry for "Downtown" from its leaf node, the leaf becomes empty. Since, in our example, $n = 3$ and $0 < \lceil (n-1)/2 \rceil$, this node must be eliminated from the B$^+$-tree. To delete a leaf node,

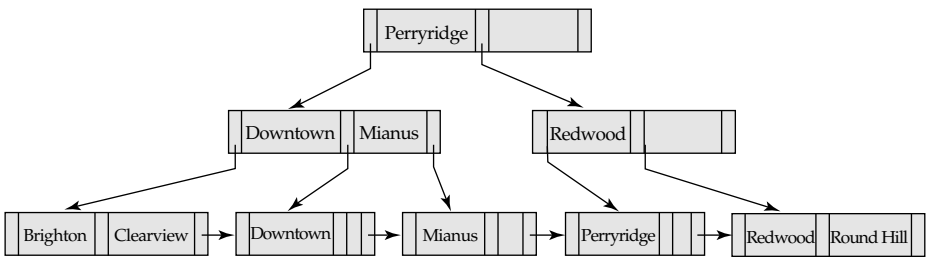


Figure 12.12 Insertion of "Clearview" into the B$^+$-tree of Figure 12.8.

```
procedure insert(value V, pointer P)
    find the leaf node L that should contain value V
    insert_entry(L, V, P)
end procedure

procedure insert_entry(node L, value V, pointer P)
    if (L has space for (V, P))
        then insert (V, P) in L
        else begin /* Split L */
            Create node L'
            Let V' be the value in L.K_1, ..., L.K_{n-1}, V such that exactly
                ⌈n/2⌉ of the values L.K_1, ..., L.K_{n-1}, V are less than V'
            Let m be the lowest value such that L.K_m ≥ V'
            /* Note: V' must be either L.K_m or V */
            if (L is a leaf) then begin
                move L.P_m, L.K_m, ..., L.P_{n-1}, L.K_{n-1} to L'
                if (V < V') then insert (P, V) in L
                else insert (P, V) in L'
            end
            else begin
                if (V = V') /* V is smallest value to go to L' */
                    then add P, L.K_m, ..., L.P_{n-1}, L.K_{n-1}, L.P_n to L'
                    else add L.P_m, ..., L.P_{n-1}, L.K_{n-1}, L.P_n to L'
                delete L.K_m, ..., L.P_{n-1}, L.K_{n-1}, L.P_n from L
                if (V < V') then insert (V, P) in L
                else if (V > V') then insert (V, P) in L'
                /* Case of V = V' handled already */
            end
            if (L is not the root of the tree)
                then insert_entry(parent(L), V', L');
                else begin
                    create a new node R with child nodes L and L' and
                        the single value V'
                    make R the root of the tree
                end
            if (L) is a leaf node then begin /* Fix next child pointers */
                set L'.P_n = L.P_n;
                set L.P_n = L'
            end
        end
end procedure
```

Figure 12.13 Insertion of entry in a B$^+$-tree.

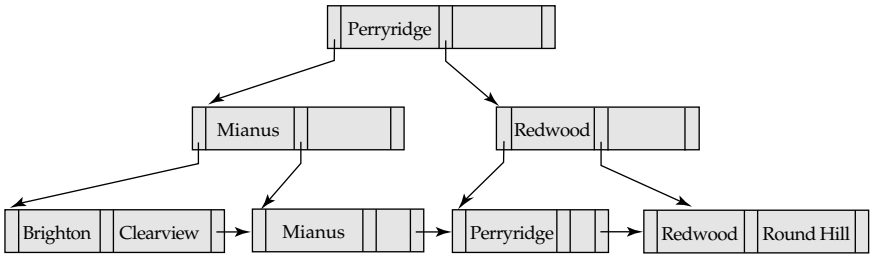


Figure 12.14 Deletion of "Downtown" from the B$^+$-tree of Figure 12.12.

we must delete the pointer to it from its parent. In our example, this deletion leaves the parent node, which formerly contained three pointers, with only two pointers. Since $2 \geq \lceil n/2 \rceil$, the node is still sufficiently large, and the deletion operation is complete. The resulting B$^+$-tree appears in Figure 12.14.

When we make a deletion from a parent of a leaf node, the parent node itself may become too small. That is exactly what happens if we delete "Perryridge" from the B$^+$-tree of Figure 12.14. Deletion of the Perryridge entry causes a leaf node to become empty. When we delete the pointer to this node in the latter's parent, the parent is left with only one pointer. Since $n = 3$, $\lceil n/2 \rceil = 2$, and thus only one pointer is too few. However, since the parent node contains useful information, we cannot simply delete it. Instead, we look at the sibling node (the nonleaf node containing the one search key, Mianus). This sibling node has room to accommodate the information contained in our now-too-small node, so we coalesce these nodes, such that the sibling node now contains the keys "Mianus" and "Redwood." The other node (the node containing only the search key "Redwood") now contains redundant information and can be deleted from its parent (which happens to be the root in our example). Figure 12.15 shows the result. Notice that the root has only one child pointer after the deletion, so it is deleted and its sole child becomes the root. So the depth of the B$^+$-tree has been decreased by 1.

It is not always possible to coalesce nodes. As an illustration, delete "Perryridge" from the B$^+$-tree of Figure 12.12. In this example, the "Downtown" entry is still part of the tree. Once again, the leaf node containing "Perryridge" becomes empty. The parent of the leaf node becomes too small (only one pointer). However, in this example, the sibling node already contains the maximum number of pointers: three. Thus, it cannot accommodate an additional pointer. The solution in this case is to **redistribute** the pointers such that each sibling has two pointers. The result appears in

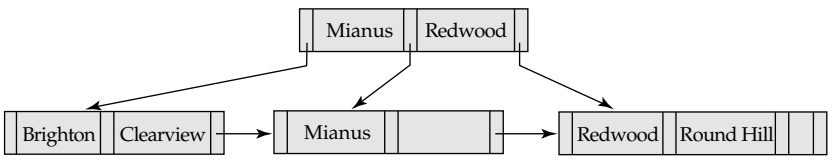


Figure 12.15 Deletion of "Perryridge" from the B$^+$-tree of Figure 12.14.